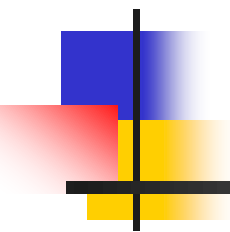


DATA STRUCTURES AND ALGORITHMS



Lecture Notes 7

Pradit Pitaksathienkul



ROAD MAP

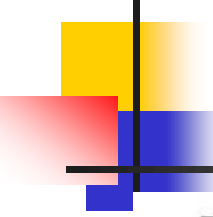
- **HASHING**
 - **General Idea**
 - **Hash Function**
 - Separate Chaining
 - Open Addressing
 - Rehashing



Hashing

- **Hashing:** implementation of hash tables
- **hash table:** an array of elements (python ใช้ datatype list)
 - fixed size **TableSize**
- Search is performed on a part of the item: **key**
- Each key is mapped into a number ตำแหน่งในอะเรย์
 - in the range 0 to TableSize-1
 - Used as array index ตัวชี้ตำแหน่งในอะเรย์
- Mapping by **hash function** ในทางอุดมคติ
 - Simple to compute
 - Ensure that any two distinct keys get different cells เป็นไปไม่ได้

How to perform insert, delete and find operations in $O(1)$ time ?



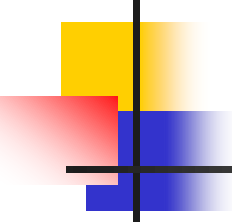
An ideal hash table

0	
1	
2	
3	john 25000
4	phil 31250
5	
6	dave 27500
7	mary 28200
8	
9	

- **Each key is mapped to a different index !**
 - **Not always possible**
 - many keys, finite indexes
- **Even distribution**

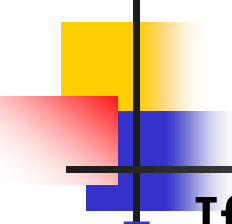
Considerations :

- Choose a hash function
- Decide what to do when two keys hash to the same value
- Decide on table size



Hash function

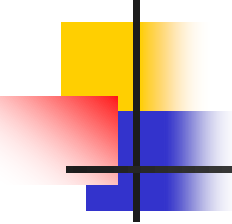
- If keys are integers
 - hash function return ***Key*** mod ***TableSize***
 - Ex: TableSize = 10
Keys = 120, 330, 1000
 - ***TableSize*** should be prime จำนวนเฉพาะ
 - ตัวเลขที่มีตัวมันเอง กับ 1 ที่หารลงตัว เท่านั้น เช่น
 - 3 , 1 , 7 , 13 , 17, 19 , 9



Hash function

- If keys are strings ข้อความ ตัวอักษรหลายตัวอยู่ติดกัน เช่น ชื่อคน
 - Add ASCII values of the characters
 - If *TableSize* is large and number of characters is small
 - **TableSize = 10000 & number of characters in a key = 8**
 $127 * 8 = 1016 < 10000$

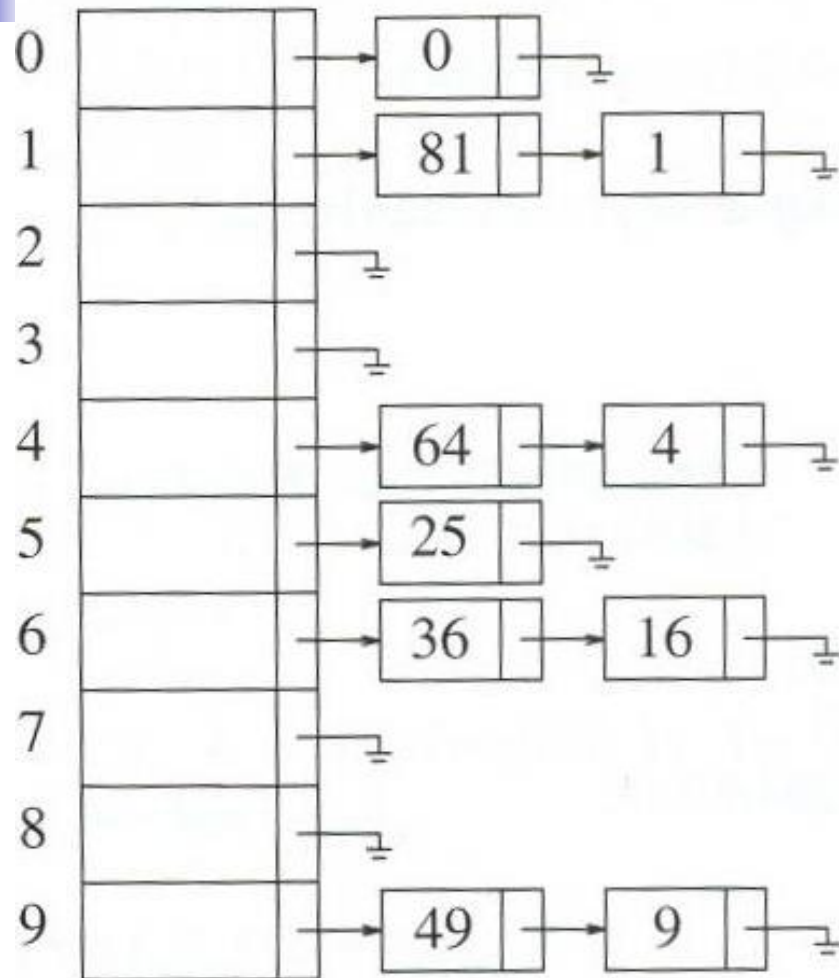
```
def hash(key, table_size):  
    hash_val = 0  
    for char in key:  
        hash_val += ord(char)  
    return hash_val % table_size
```



Collision การชนกัน

- Main programming detail is **collision resolution** การแก้การชนกัน
- If when an element is inserted, it hashes to the same value as an already inserted element, there is ***collision***.
- There are several methods to deal with this problem
 - Separate แยก chaining ต่อกันเป็นโซ่
 - Open addressing

Separate Chaining Hash Table



- Keep a list of all elements that hash to the same value
- TableSize = 10
 - **is not good**
 - not prime



Open Addressing

If collision $\rightarrow$ try an alternate cell

$h_0(x), h_1(x), h_2(x), \dots$

$h_i(x) = (\text{hash}(x) + F(i)) \bmod \text{TableSize}$

$F(0) = 0$

Linear Probing

- **F** is a linear function of **I** สมการเชิงเส้น ไม่มีเลขยกกำลัง
 - $F(i) = i$ ง่ายที่สุด

	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1					58	58
2						69
3						
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

Insert keys

{89, 18, 49, 58, 69}

- When 49 is inserted collision occurs
 - Put into the next available spot 0
- 58 collidates with 18, 89, 49



Linear Probing

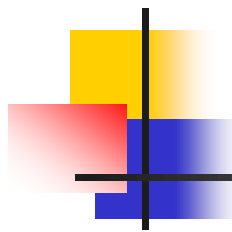
- Problem: It is **not easy** to **delete** an element
 - May have caused a collision before
 - Mark the element deleted
- Problem: Primary Clustering การเกาะกลุ่ม



Quadratic Probing

- $F(i)$ is a quadratic function เลขยกกำลัง

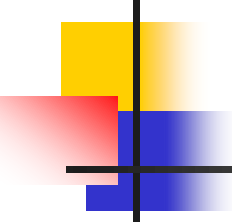
$$\text{Ex : } F(i) = i^2$$



Quadratic Probing

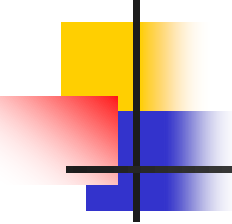
	Empty Table	After 89	After 18	After 49	After 58	After 69
0				49	49	49
1						
2					58	58
3						69
4						
5						
6						
7						
8			18	18	18	18
9		89	89	89	89	89

- When 49 collides with 89, next position attempted is one cell away
- 58 collides at position 8. The cell one away is tried, another collision occurs. It is inserted into the cell $2^2=4$ away



Quadratic Probing

- Solves primary clustering problem
- All empty cells may not be accessed
 - A loop around full cells may happen
 - Hash table not full but empty space not found
- Problem : Secondary clustering!...



Double Hashing

- Use second hash function ฟังก์ชันแฮช ที่สอง
- $F(i) = i * \mathbf{hash_2(x)}$ ฟังก์ชันแฮช ที่สอง ของ X
- Poor example :
 $hash_2(x) = X \bmod 9$
 $hash_1(x) = X \bmod 10$
TableSize = 10

If $X = 99$ what happens ?

$hash_2(x) \neq 0$ for any X



Double Hashing

- Good choice :

$$\text{hash}_2(x) = R - (X \bmod R)$$

R is a prime and $< \text{TableSize}$

R เป็นเลขจำนวนเฉพาะที่มากที่สุด แต่น้อยกว่า TableSize

ถ้า TableSize เป็น 10 , R จะเท่ากับ 7 ทำไมไม่เป็น 8,9

ถ้า TableSize เป็น 11 R จะเท่ากับ 7



Double Hashing

	Empty Table	After 89	After 18	After 49	After 58	After 69
0						69
1						
2						
3					58	58
4						
5						
6				49	49	49
7						
8			18	18	18	18
9		89	89	89	89	89

$$\text{hash}_2(x) = 7 - (x \bmod 7)$$



Rehashing

- If the elements in the table gets too large, number of probes increases.
- Running time of operations **starts taking too long** and **insertions might fail**
- Solution : Rehashing with larger TableSize (usually *2)
- When to rehash
 - if the elements in the table > 0.5
 - if more insertion fails



Rehashing Example

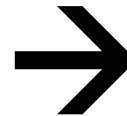
- Elements 13, 15, 24 and 6 is inserted into an open addressing hash table of size 7
- $H(X) = X \bmod 7$
- Linear probing is used to resolve collisions

0	6
1	15
2	
3	24
4	
5	
6	13

Rehashing Example

- If 23 is inserted, the table is over **70** percent full.

0	6
1	15
2	23
3	24
4	
5	
6	13



A new table is created

17 is the first prime twice as large as the old one; so

$$H_{\text{new}}(X) = X \bmod 17$$

0	
1	
2	
3	
4	
5	
6	6
7	23
8	24
9	
10	
11	
12	
13	13
14	
15	15
16	